

## EXPERIMENTS: 11 TO 20

### Exp-11

```
1 #include <stdio.h>
2 #include <pthread.h>
3
4 void *thread1(void *arg)
5 {
6     printf("Thread 1 is running\n");
7     for (int i = 1; i <= 5; i++)
8         printf("Thread 1: %d\n", i);
9     return NULL;
10 }
11
12 void *thread2(void *arg)
13 {
14     printf("Thread 2 is running\n");
15     for (int i = 1; i <= 5; i++)
16         printf("Thread 2: %d\n", i);
17     return NULL;
18 }
19
20 int main()
21 {
22     pthread_t t1, t2;
23
24     pthread_create(&t1, NULL, thread1, NULL);
25     pthread_create(&t2, NULL, thread2, NULL);
26
27     pthread_join(t1, NULL);
28     pthread_join(t2, NULL);
29
30     printf("Both threads completed.\n");
31
32     return 0;
33 }
```

```
Thread 1: 1
Thread 1: 2
Thread 2 is running
Thread 2: 1
Thread 2: 2
Thread 2: 3
Thread 2: 4
Thread 2: 5
Thread 1: 3
Thread 1: 4
Thread 1: 5
Both threads completed.

...Program finished with exit code 0
Press ENTER to exit console
```

### Exp-12

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <semaphore.h>
4 #include <unistd.h>
5
6 #define N 5
7
8 sem_t chopstick[N];
9
10 void *philosopher(void *arg)
11 {
12     int id = *(int *)arg;
13
14     printf("Philosopher %d is thinking\n", id);
15     sleep(1);
16
17     sem_wait(&chopstick[id]);
18     sem_wait(&chopstick[(id + 1) % N]);
19
20     printf("Philosopher %d is eating\n", id);
21     sleep(2);
22
23     sem_post(&chopstick[(id + 1) % N]);
24     sem_post(&chopstick[id]);
25
26     printf("Philosopher %d finished eating\n", id);
27
28     return NULL;
29 }
30
31 int main()
32 {
33     pthread_t threads[N];
34     int id[N];
35 }
```

```
Philosopher 0 is thinking
Philosopher 3 is thinking
Philosopher 1 is thinking
Philosopher 4 is thinking
Philosopher 2 is eating
Philosopher 4 is eating
Philosopher 4 finished eating
Philosopher 2 finished eating
Philosopher 1 is eating
Philosopher 3 is eating
Philosopher 3 finished eating
Philosopher 0 is eating
Philosopher 0 finished eating
Philosopher 1 finished eating
```

## Exp-13

```

First Fit:
Process 1 -> Block 2
Process 2 -> Block 5
Process 3 -> Block 2
Process 4 -> Not Allocated

Best Fit:
Process 1 -> Block 4
Process 2 -> Block 2
Process 3 -> Block 3
Process 4 -> Block 5

Worst Fit:
Process 1 -> Block 5
Process 2 -> Block 2
Process 3 -> Block 5
Process 4 -> Not Allocated

...Program finished with exit code 0
Press ENTER to exit console.

```

## Exp-14

```

--- Single Level Directory ---
1. Create File
2. Display Files
3. Search File
4. Delete File
5. Exit
Enter choice: 1
Enter file name: ffff
File created successfully.

--- Single Level Directory ---
1. Create File
2. Display Files
3. Search File
4. Delete File
5. Exit
Enter choice: 3
Enter file name to search: de
File not found.

--- Single Level Directory ---
1. Create File
2. Display Files
3. Search File
4. Delete File
5. Exit
Enter choice: 1

```

## Exp-15

```
1 #include <stdio.h>
2 #include <string.h>
3
4 #define MAX_USERS 10
5 #define MAX_FILES 20
6
7 struct User
8 {
9     char username[30];
10    char files[MAX_FILES][30];
11    int fileCount;
12 };
13
14 int main()
15 {
16     struct User users[MAX_USERS];
17     int userCount = 0;
18     int choice;
19     char username[30], filename[30];
20
21     while (1)
22     {
23         printf("\n--- Two Level Directory ---\n");
24         printf("1. Create User\n");
25         printf("2. Create File\n");
26         printf("3. Display Directory\n");
27         printf("4. Search File\n");
28         printf("5. Exit\n");
29         printf("Enter choice: ");
30         scanf("%d", &choice);
31
32         if (choice == 1)
33         {
34             printf("Enter username: ");
35             scanf("%s", username);
36
37             --- Two Level Directory ---
38             1. Create User
39             2. Create File
40             3. Display Directory
41             4. Search File
42             5. Exit
43             Enter choice: 1
44             Enter username: yathra
45             User created successfully.
46
47             --- Two Level Directory ---
48             1. Create User
49             2. Create File
50             3. Display Directory
51             4. Search File
52             5. Exit
53             Enter choice: 5
54
55             ...Program finished with exit code 0
56             Press ENTER to exit console.
```

## Exp-16

```
1 #include <stdio.h>
2
3 struct Employee
4 {
5     int id;
6     char name[30];
7     float salary;
8 };
9
10 int main()
11 {
12     FILE *fp;
13     struct Employee emp;
14     int n, position;
15
16     fp = fopen("employee.dat", "wb+");
17
18     if (fp == NULL)
19     {
20         printf("Unable to create file.\n");
21         return 1;
22     }
23
24     printf("Enter number of employees: ");
25     scanf("%d", &n);
26
27     for (int i = 0; i < n; i++)
28     {
29         printf("\nEnter Employee %d details\n", i + 1);
30
31         printf("ID: ");
32         scanf("%d", &emp.id);
33
34         printf("Name: ");
35         scanf("%s", emp.name);
36
37         Enter Employee 1 details
38         ID: ss
39         Name: Salary: 44444
40
41         Enter Employee 2 details
42         ID: tt
43         Name: Salary: 44444
44
45         Enter employee position to access:
```

## Exp-17

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, m;
6
7     printf("Enter number of processes: ");
8     scanf("%d", &n);
9
10    printf("Enter number of resources: ");
11    scanf("%d", &m);
12
13    int allocation[n][m];
14    int max[n][m];
15    int need[n][m];
16    int available[m];
17    int finish[n];
18    int safeSequence[n];
19
20    printf("\nEnter Allocation Matrix:\n");
21
22    for (int i = 0; i < n; i++)
23        for (int j = 0; j < m; j++)
24            scanf("%d", &allocation[i][j]);
25
26    printf("\nEnter Maximum Matrix:\n");
27
28    for (int i = 0; i < n; i++)
29        for (int j = 0; j < m; j++)
30            scanf("%d", &max[i][j]);
31
32    printf("\nEnter Available Resources:\n");
33
34    for (int j = 0; j < m; j++)
35        scanf("%d", &available[j]);
```

```
Enter number of resources: 3
Enter Allocation Matrix:
122334
22334
33344
33444
```

## Exp-18

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <semaphore.h>
4 #include <unistd.h>
5
6 #define SIZE 5
7
8 int buffer[SIZE];
9 int in = 0, out = 0;
10
11 sem_t empty;
12 sem_t full;
13 pthread_mutex_t mutex;
14
15 void *producer(void *arg)
16 {
17     for (int i = 1; i <= 10; i++)
18     {
19         sem_wait(&empty);
20         pthread_mutex_lock(&mutex);
21
22         buffer[in] = i;
23         printf("Producer produced: %d\n", i);
24
25         in = (in + 1) % SIZE;
26
27         pthread_mutex_unlock(&mutex);
28         sem_post(&full);
29
30         sleep(1);
31     }
32
33     return NULL;
34 }
```

```
Consumer consumed: 1
Producer produced: 2
Consumer consumed: 2
Producer produced: 3
Producer produced: 4
Consumer consumed: 3
Producer produced: 5
Producer produced: 6
Consumer consumed: 4
Producer produced: 7
Producer produced: 8
Consumer consumed: 5
Producer produced: 9
Producer produced: 10
Consumer consumed: 6
Consumer consumed: 7
Consumer consumed: 8
Consumer consumed: 9
```

## Exp-19

```
1 #include <stdio.h>
2 #include <pthread.h>
3
4 int counter = 0;
5
6 pthread_mutex_t lock;
7
8 void *increment(void *arg)
9 {
10     for (int i = 0; i < 100000; i++)
11     {
12         pthread_mutex_lock(&lock);
13
14         counter++;
15
16         pthread_mutex_unlock(&lock);
17     }
18     return NULL;
19 }
20
21 int main()
22 {
23     pthread_t t1, t2;
24
25     pthread_mutex_init(&lock, NULL);
26
27     pthread_create(&t1, NULL, increment, NULL);
28     pthread_create(&t2, NULL, increment, NULL);
29
30     pthread_join(t1, NULL);
31     pthread_join(t2, NULL);
32
33     printf("Final Counter Value = %d\n", counter);
34
35 }
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```

## Exp-20

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <semaphore.h>
4 #include <unistd.h>
5
6 int data = 0;
7 int readers = 0;
8
9 sem_t mutex;
10 sem_t writeLock;
11
12 void *reader(void *arg)
13 {
14     int id = *(int *)arg;
15
16     sem_wait(&mutex);
17     readers++;
18
19     if (readers == 1)
20         sem_wait(&writeLock);
21
22     sem_post(&mutex);
23
24     printf("Reader %d is reading data = %d\n", id, data);
25     sleep(1);
26
27     sem_wait(&mutex);
28     readers--;
29
30     if (readers == 0)
31         sem_post(&writeLock);
32
33     sem_post(&mutex);
34
35     return NULL;
36 }
```

```
Reader 3 is reading data = 0
Reader 2 is reading data = 0
Writer 1 wrote data = 1
Writer 2 wrote data = 2
```

```
...Program finished with exit code 0
Press ENTER to exit console.
```